

PhD Application Assignment

Distributed and Parallel Systems Group,
University of Innsbruck

August 2024

1 Introduction

Thank you for your interest in the PhD position at the Distributed and Parallel Systems Group, University of Innsbruck. As part of the application process, we request you complete the assignment outlined in this document. The assignment consists of two main components:

1. **Theoretical Task:** Answer questions related to algorithms and methods in distributed systems.
2. **Practical Task:** Implement a runtime system and develop a distributed application.

Please submit your solutions as follows:

- Provide a PDF report addressing and justifying your answers for both tasks.
- Provide us (GitHub user *dps-uibk*) access to a private GitHub repository containing your runtime system implementation and a complete application description.
- Package the PDF report and a README file (which outlines the ZIP file contents) into a ZIP file.

The report and implementation will be reviewed in detail during a follow-up interview, should your initial evaluation be positive. This assignment aims to assess your creativity, problem-solving skills, and theoretical knowledge in the context of distributed systems. It is designed to challenge you to demonstrate innovative thinking and technical expertise. We encourage you to iteratively refine your solutions, as developing high-quality, robust answers often requires time and thoughtful improvement.

Please prioritize Task 2 over Task 1; if you need to choose, focus more on providing a comprehensive solution for Task 2, even if it means giving a less detailed solution for Task 1.

Avoid using AI tools like ChatGPT to generate your answers. AI-generated texts do not meet the depth and detail we expect from applicants. Additionally, as your report will be discussed in a subsequent interview, we recommend thoroughly studying the subjects to ensure you have the necessary knowledge.

2 Task 1: Theoretical Questions

Answer the questions below, summarizing your understanding of the subjects in full detail. Your responses should go beyond readily available internet information, incorporating insightful observations based on your knowledge. Write your text with the depth and rigor appropriate for summarizing a scientific study. Include your answers in your report.

1. Concurrency and Parallelism in Programming Languages:

- (a) Examine how different programming languages (e.g., Java and Python) handle concurrency and parallelism.
- (b) Analyze how these concurrency features impact the design and implementation of distributed systems. Provide examples where applicable.

2. Leader Election Algorithms:

- (a) Describe an efficient algorithm for leader election in a distributed system. Include a discussion of its time complexity and space complexity.
- (b) Explain why the algorithm is effective and under what conditions it performs best.

3. High Availability and Fault Tolerance Best Practices:

- (a) Discuss best practices for ensuring high availability and fault tolerance in software systems. Include considerations for design, testing, and monitoring.
- (b) Provide examples of strategies to achieve these goals and their impact on system reliability.

4. Communication Strategy for Isolated Networks:

- (a) Design a strategy for enabling communication between resources in Private Network A and Private Network B, where direct external connections are impossible.
- (b) Consider existing mechanisms such as proxies, VPNs, relay services, or novel strategies and explain how your solution ensures secure and reliable communication.

5. Understanding of the Computing Continuum:

- (a) Provide a detailed description of the Computing Continuum¹.
- (b) Address the purpose of each continuum layer and give examples of common use cases. Address aspects such as the characteristics of resources in each layer.

¹Cloud-Edge-IoT

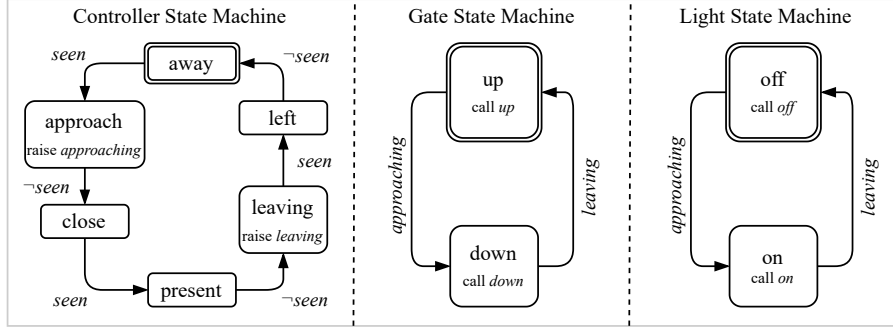


Figure 1: Three state machines constitute the application described by the state machine model. The controller state machine monitors the position of a train using sensors located before, at, and after a railway crossing, which generate *seen* and $\neg$ *seen* events.

3 Task 2: Design and Implement a Distributed System

3.1 Problem Statement

We model a distributed system as a network of distributed state machines. The following components define each state machine:

Function Each state machine has a set of functions that can be called that process input data and produce output data.

Event Each state machine can respond to events E ; an event can carry associated data.

Action Each state machine has a set of actions A . Two action types exist: one type of action raises an event, and the other calls a function.

State Each state machine consists of a set of states S . State transitions occur based on specific events, and actions are executed when entering a state (Moore semantics), i.e., the output function is $\omega : S \rightarrow 2^A$.

Transition Transitions T in a state machine are defined as a subset of $S \times E \times S$. Each transition $t = (x, e, y)$ describes a change from a source state $x \in S$ to a target state $y \in S$, triggered by an event $e \in E$.

Your task is to design and implement a distributed state machine runtime environment with the following capabilities:

- Support the execution of one or more state machines.
- Facilitate interaction between state machines through the exchange of events.
- Allow state machines to invoke functions.

Functions can be serverless functions or external services.

3.2 Part 1: Model

Design a state machine model capable of representing any arbitrary state machine according to the specifications provided in the previous section. The model should include all necessary components.

1. Document your model in detail in your report.
 - (a) Explain the reasoning behind your design choices.
 - (b) Explore potential extensions to enhance the model’s functionality or applicability.

This state machine model will be utilized in Part 2 to describe a distributed application involving multiple state machines. Ensure that the model is versatile enough to describe any state machine and consider how it can be serialized for use in Part 3. An example of a state machine markup language is SCXML [1].

3.3 Part 2: Application

Apply your state machine model to the application scenario outlined in Section 3.3.1, which involves several interacting state machines.

1. Implement the functions required to support the application’s needs.
2. Use your state machine model to describe the application, which consists of multiple distributed state machines.
3. Document the application description and the implemented functions in your report.

3.3.1 Application Problem Statement

This scenario involves a state machine system managing lights and gates at a railroad crossing, inspired by Leveson’s Petri Net approach to safety-critical systems [2]. Figure 1 illustrates the state machines for the *controller*, *gate*, and *light*.

- The *controller* state machine responds to *seen* and $\neg$ *seen* events from sensors placed around the crossing.
- The *seen* event is triggered when a sensor detects a train, while $\neg$ *seen* indicates the absence of a train. Sensors are positioned before, at, and after the crossing to monitor the train’s movement.
- The *controller* state machine processes sequences of *seen* and $\neg$ *seen* events to track the train’s approach and departure.
- The *gate* and *light* state machines manage the physical components based on signals from the *controller*. They respond to *approaching* and *leaving* events to control the gates and lights, invoking services to raise or lower the gates and switch the lights on or off.

Use the model you created in Part 1 to describe this system. Ensure that the description is suitable for use in Part 3.

- In your report, explain how each component of the application is represented in your state machine model.
- Include diagrams illustrating state transitions and interactions among components.
- Justify your design choices and discuss how the state machine model supports the deployment of this application across the Computing Continuum.
- Suggest potential enhancements to improve the model’s scalability, adaptability, or efficiency in different scenarios.

The application simulates the operation of gates and lights, including their activation and deactivation.

3.4 Part 3: Runtime

Implement the runtime environment in Java to support the execution of the **distributed** application across **at least three resources**, one for each state machine. Use an orchestration tool such as Docker Swarm, Nomad, or Consul². Ensure that the state machine behavior is **not hard-coded**; your runtime environment should utilize the state machine model from Part 1 and the application description from Part 2 to manage and execute the state machines on the distributed resources.

- In your report, provide a detailed description of your implementation and orchestration environment. Explain how the runtime environment manages and coordinates the state machines.
- Document your codebase thoroughly to support understanding and maintenance. Include instructions for executing and deploying the system and any necessary configuration details.
- Submit the Java implementation with well-documented code and a Docker image of the runtime environment.

Ensure your documentation is clear and comprehensive, covering all aspects needed to effectively understand, deploy, and operate the distributed application.

References

- [1] Jim Barnett. Introduction to SCXML. In Deborah A. Dahl, editor, *Multimodal Interaction with W3C Standards: Toward Natural User Interfaces to Everything*, pages 81–107. Springer International Publishing, Cham, 2017.
- [2] N.G. Leveson and J.L. Stolzy. Safety Analysis Using Petri Nets. *IEEE Transactions on Software Engineering*, SE-13(3):386–397, March 1987.

²Provide a classical distributed application in case these tools are not available in your country